ECE 2036 Lab 2: Basic Class Construction and Use: Complex Numbers with Simple Applications

Due: Friday, February 13, 11:59 PM

In this lab, you will: 1) get experience with class creation through implementation of a complex number class, and 2) explore some simple applications of complex numbers.

Important note: I have put some files to get you started with the assignment in the course's shared directory on ICE, which is `/storage/ice-shared/ece2036a`. The files are in the `Lab2_files` sub-directory of this shared space. Create a Lab2 directory inside your home directory on ICE and copy these files into it – do ***not*** add or delete anything in the shared space!! To do this, type the following while in your home directory (after you've created the Lab2 directory):

```
cp   /storage/ice-shared/ece2036a/Lab2_files/*   Lab2
```

Part 1: Implementing a complex number class

Note: There is a complex number class within the C++ standard library (std) that implements some, but not all, of the features described here. For this lab, you must implement your own class for complex numbers and not use the standard library complex class at all.

The shared directory has a file named `complex.h`, which defines the interface for your complex number class. The contents of this file are shown below:

```
class Complex
{
public:
  Complex();                              // default constructor, initialize to 0.0 + 0.0i
  Complex(double, double);
  double getReal() const;
  void setReal(double);
  double getImag() const;
  void setImag(double);
  Complex add(Complex) const;             // add this complex num to complex parameter, return result
  Complex subtract(Complex) const;        // this complex num minus complex parameter, return result
  Complex multiply(Complex) const;        // multiply this complex num times complex parameter, return result
  Complex divide(Complex) const;          // this complex num divided by complex parameter, return result
  Complex conjugate() const;              // conjugate of a + bi is a - bi
  double norm() const;                    // a^2 + b^2
  double magnitude() const;               // sqrt(a^2 + b^2)
  void outputAsString() const;

private:
  double real;
  double imag;
};
```

In this part, you must implement all of the member functions of the Complex class listed above. The class has two constructors: if no arguments are provided when instantiating a Complex number

object, the real and imaginary variables of the object should both be set to 0.0; however, if two arguments, e.g. *a* and *b*, are provided when instantiating an object, it will be initialized to *a* + *bi.* There are getter and setter functions for accessing the real and imaginary parts of a Complex number, functions for performing addition, subtraction, multiplication, and division of complex numbers, and functions to return the conjugate, norm, and magnitude of a complex number. Finally, there is a function that will output a complex number as a string in the form *a* + *bi.* I have provided starter code in the file complex.cc in the shared directory, which has an implementation of the default constructor function for the class. You should add implementations of all the remaining member functions to the complex.cc file.

Once you have implemented your class, you can test it using the test.cc file in the shared directory. You can compile the test program with the Complex class using the following command:

```
g++ test.cc complex.cc
```

When this command compiles with no errors, it will produce an executable file, which when run, should produce the following output:

```
c1 = -4.96 + 21.67i

c2 = 17.4 + -4.81i

c1 + c2 = 12.44 + 16.86i

c1 - c2 = -22.36 + 26.48i

c1 * c2 = 17.9287 + 400.916i

c1 / c2 = -0.584655 + 1.08378i

conjugate of c1 is -4.96 + -21.67i

magnitude of c2 is 18.0526

norm of c2 is 325.896

copying c1 into c8 yields: -4.96 + 21.67i
```

If the test program compiles and produces the above output, your Complex class implementation is correct – congratulations!

Part 2: Finding (possibly complex) roots of a quadratic equation

In this part, you will write a C++ program to calculate the roots of a quadratic equation, which are given by:

$$x1, x2 = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Note that if $b^2 - 4ac$ is negative, then the roots of the equation are complex numbers.

To calculate one root, your code must use a global function that has four parameters: a, b, c, and a flag for plus or minus, and returns the root as a Complex number. In your main program, prompt the user to input double values for *a*, *b*, and *c* and call your global function twice with those input values to find the two roots. Output the two root values as complex numbers using the outputAsString() member function from your Complex class. Make sure to add `#include "complex.h"` at the top of your program!

Sample outputs for two values of *a*, *b*, and *c* are:

```
Enter a, b, c:    1 10 10

roots are:

        -1.12702 + 0i
        -8.87298 + 0i


Enter a, b, c:    1 10 100

roots are:

        -5 + 8.66025i
        -5 + -8.66025i
```

In your Lab 2 directory, name your program `lab2part2.cc`. Compile your code with a similar command as in Part 1:

```
g++ lab2part2.cc complex.cc
```

Part 3: Calculating the current in an RLC series circuit

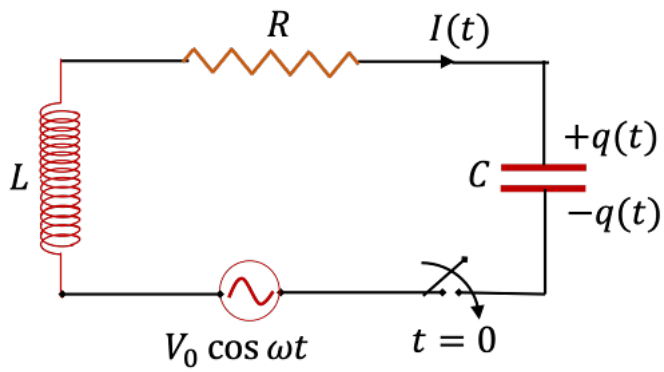
The figure below shows a circuit with an AC voltage source, a resistor, an inductor, and a capacitor. The current in an RLC circuit is given by V_0/Z, where $Z = Z_R + Z_L + Z_C$. The individual impedances are as follows:

$Z_R = R$,

$Z_L = \omega Li$

$Z_C = (-1/(\omega C))i$

Note that the inductor and capacitor impedances are complex numbers! The current in Amperes is given by the magnitude of the complex number V_0/Z.



Using your Complex class, write code to calculate and output the current in an RLC series AC circuit. You should have the user input R, L, C, V_0, and f, where f is the AC frequency. (Note that $\omega = 2\pi f$.) As in Part 2, make sure to add #include "complex.h" at the top of your program! In your Lab 2 directory, name your program lab2part3.cc. Compile your code with a similar command as in Parts 1 and 2:

```
g++ lab2part3.cc complex.cc
```

A sample program output for R = 20 Ohms, L = 0.15 Henrys, C = 10^{-4} Farads, V_0 = 120 Volts, and f = 60 Hz is:

```
Enter R, L, C, V, f:    20  0.15  0.0001   120   60

current is 3.32647 amps
```

Turn-In Instructions

You will compress and bundle your entire directory on ICE and turn this in on Canvas.

1. Create a directory under your home directory named Lab2.
2. Make sure the following files are all in your Lab2 directory:
   - complex.h (file given with assignment)
   - complex.cc (your implementation of class Complex)
   - test.cc (test program for Part 1, given with assignment)
   - lab2part2.cc (your program to calculate possibly complex roots of a quadratic eq'n)
   - lab2part3.cc (your program to calculate the current in an RLC series circuit)
3. From your home directory, type the following command:

   ```
   tar  -cvzf   <yourusername>Lab2.tar.gz   ./Lab2
   ```

4. Submit the file <yourusername>Lab2.tar.gz via Canvas. (As detailed in Lab 1, it is ***your*** responsibility to make sure that your tarball contains the proper files.)

APPENDIX A: ECE 2036 Lab Grading Rubric

If a student's program runs correctly and produces the desired output, the student has the potential to get a 100 on his or her lab; however, TA's will **randomly** look through this set of "perfect-output" programs to look for other elements of meeting the lab requirements. The table below shows typical deductions that could occur.

In addition, if a student's code does not compile on `coc-ice.pace.gatech.edu`, then they will have an automatic 30% deduction on the lab. Code that compiles but does not match the sample output can incur a deduction from 10% to 30% depending on how poorly the output matches the output specified by the lab. This is in addition to the other deductions listed below or due to the student not attempting the entire assignment.

AUTOMATIC GRADING POINT DEDUCTIONS

Element	Percentage Deduction	Details
Does Not Compile on ICE Cluster	30%	Part 1 or Part 2 program does not compile on ICE cluster! (15% for each part)
Output Incorrect	10%-30%	Your output should approximately match the solution's output for Part 1 test.cc and for Parts 2, 3 with random test inputs.

ADDITIONAL GRADING POINT DEDUCTIONS

Element	Percentage Deduction	Details
Is not turned in with a tarball	10%	Just to be clear on the submission
Code correctness	70%	The TA will inspect your code to make this determination.
Clear Self-Documenting Coding Styles	5%-15%	This can include no indentation, using unclear variable names, unclear comments, or compiling with warnings. (see Appendix B)

LATE POLICY

Element	Percentage Deduction	Details
Late Deduction	See details	< 2 hours late: 5 points off total > 2 hours late, up to 1 day late: 10 points off total > 1 day late, up to 2 days late: 20 points off total 20 additional points off for each day late beyond 2 days